EXPERIMENT-9

Name: Shasmeen Syed ROLL NO:240701492

Aim: To design input forms that validate data, such as email and phone number, and display error messages using HTML/CSS and JavaScript with Validator.js.

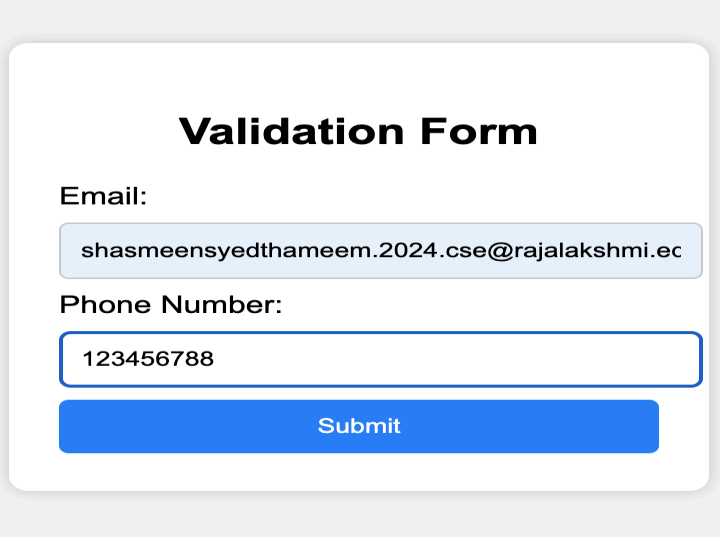
Step 1:

Code

```
<!DOCTYPE html>
<html>
<head>
   <title>Form Validation</title>

   <!-- Validator.js CDN -->
   <script
src="https://cdnjs.cloudflare.com/ajax/libs/validator/13.9.0/validator.min.js"></script>
</head>
<body>

<h2>Validation Form</h2>

<form id="form">
   <label>Email:</label><br>
   <input type="text" id="email"><br>
   <span id="emailError"></span><br><br>

   <label>Phone:</label><br>
   <input type="text" id="phone"><br>
   <span id="phoneError"></span><br><br>

   <button type="submit">Submit</button>
</form>

<script>
document.getElementById("form").addEventListener("submit", function(e) {
   e.preventDefault();

   let email = document.getElementById("email").value;
   let phone = document.getElementById("phone").value;

   let emailError = document.getElementById("emailError");
   let phoneError = document.getElementById("phoneError");
```

```
    emailError.textContent = "";
    phoneError.textContent = "";

    let valid = true;

    if (!validator.isEmail(email)) {
        emailError.textContent = "Invalid email";
        valid = false;
    }

    if (!validator.isMobilePhone(phone, "en-IN")) {
        phoneError.textContent = "Invalid phone number";
        valid = false;
    }

    if (valid) {
        alert("Form submitted successfully");
    }
});
</script>

</body>
</html>
```

Output:



Explanation

In the HTML file, you build the basic structure of the form. You include a `<form>` element with input fields for email and phone number, each having a unique `id`. These IDs are essential because JavaScript will use them to access and validate user input. Below each input field, you add a `<div>` to display error messages dynamically. You also link your CSS file using the `<link>` tag and include the Validator.js library through a CDN. This library provides ready-made functions like `isEmail()` and `isMobilePhone()`, which save time and reduce complexity compared to writing validation logic from scratch.

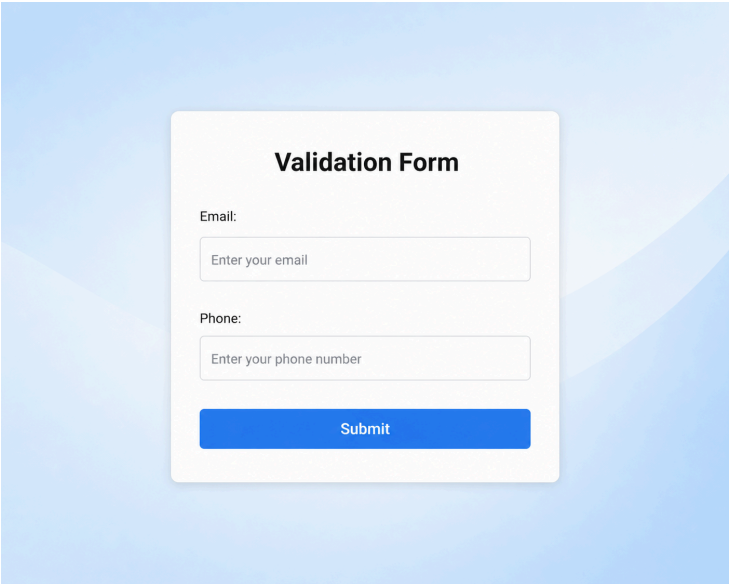
Step 2:

Code

```
/* Background */
body {
    font-family: Arial, sans-serif;

    /* Soft gradient background */
    background: linear-gradient(to bottom right, #dfe9f3, #ffffff);

    padding: 40px;
}

/* Heading */
h2 {
    text-align: center;
}

/* Form box */
form {
    width: 300px;
    margin: auto;
    background: white;
    padding: 20px;
    border-radius: 6px;
    box-shadow: 0 0 10px rgba(0,0,0,0.15);
}

/* Input fields */
input {
    width: 100%;
    padding: 8px;
    margin: 8px 0;
    border: 1px solid #ccc;
    border-radius: 4px;
}

/* Error text */
span {
    color: red;
    font-size: 12px;
}

/* Submit button */
button {
    width: 100%;
    padding: 10px;
    background: #007bff;
    border: none;
    color: white;
    border-radius: 4px;
    cursor: pointer;
}
```

```
button:hover {
    background: #0056b3;
}
```

Output



Explanation

The CSS file is used to enhance the appearance of the form. A gradient background is applied using `linear-gradient`, which gives a modern and visually appealing look. Flexbox is used to center the form both vertically and horizontally on the screen. The form container is styled with padding, rounded corners, and a shadow to make it look like a card. Input fields are styled for better usability, and special classes like `.error-input` and `.success` are defined to visually indicate validation results. When an input is invalid, its border turns red, and when it is valid, it turns green. The submit button is also styled with a gradient to match the overall theme

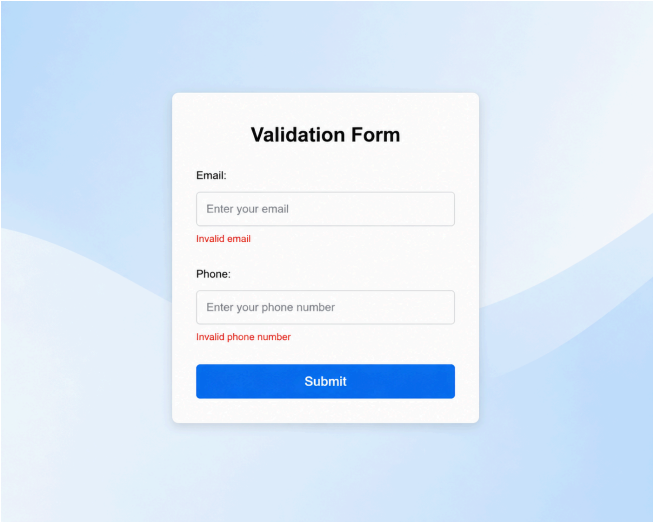
Step 3:

Code

```
const form = document.getElementById('myForm');

form.addEventListener('submit', function(e) {
  e.preventDefault();

let email = document.getElementById('email'); let
  phone = document.getElementById('phone');

let emailError = document.getElementById('emailError');
```

```
  let phoneError = document.getElementById('phoneError');

  let valid = true;

  emailError.textContent = "";
  phoneError.textContent = "";
  email.classList.remove('error-input', 'success');
  phone.classList.remove('error-input', 'success');

  if (!validator.isEmail(email.value)) {
    emailError.textContent = "Enter a valid email";
    email.classList.add('error-input');
    valid = false;
  } else {
    email.classList.add('success');
  }

  if (!validator.isMobilePhone(phone.value, 'en-IN')) {
    phoneError.textContent = "Enter a valid phone
    number"; phone.classList.add('error-input');
    valid = false;
  } else {
    phone.classList.add('success');
  }

  if (valid) {
    alert("Form Submitted Successfully!");
  }
})
 ;
```

Output



Explanation

In the JavaScript file, the validation logic is implemented. First, the form is selected using its ID, and an event listener is added to handle the submit action. The `preventDefault()` method is used to stop the form from reloading the page when submitted, allowing validation to occur smoothly. The input values and corresponding error display elements are then accessed. Before

performing validation, any previous error messages and styles are cleared to ensure a fresh check each time the user submits the form.